

# AUTOMATED SOFTWARE TESTING

## *Testing Journal & Documentation*

Quality Assurance for Final Project

### I. Personal & Group Information

---

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| Full Name        | Mheea Rose J. Andes                                                |
| Course & Section | BSIT 3C                                                            |
| Group Name       | Salespresso                                                        |
| Project Title    | Salespresso: A Web-Based E-Commerce System for Jazsam Coffee Store |
| Submission Date  | May 20, 2026                                                       |

### II. Assigned Task

---

|                                  |                                                                    |
|----------------------------------|--------------------------------------------------------------------|
| Feature / Functionality Assigned | Products API — CRUD (Create, Read, Update, Delete) & Authorization |
| Type of Test I Performed         | Functional Testing, API Testing, Authorization/Security Testing    |
| Tool / Framework I Used          | Python 3.13, pytest, requests                                      |
| Programming Language Used        | Python                                                             |
| My Test Script Filename(s)       | test_products.py                                                   |

#### Brief description of what I tested and how I approached it:

I was tasked with testing the Products API endpoint of the Salespresso system, focusing on its CRUD functionality and verifying that only authenticated admin users could perform write operations. This API plays a major role in the project since the entire store catalog relies on it, so any issue with creating or retrieving products would directly affect the customer ordering experience.

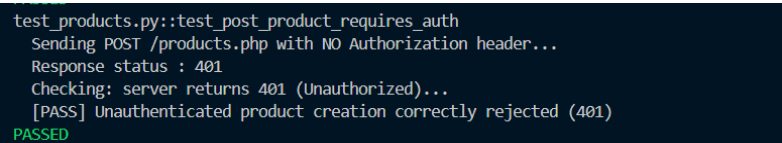
My testing approach was to first check the public read endpoint to ensure that the API was responding correctly. After that, I tested authorization enforcement by attempting write operations without an authorization token to confirm access restrictions were enforced.

Finally, I verified that authenticated admin requests worked as intended. To avoid hitting the rate limit during testing, I used a session-scoped admin token from confest.py that was shared across all tests.

### III. Test Scenario Documentation

| Test Case #01 — Get All Products (Public) |                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Functionality Tested                      | Products — GET Endpoint (Public)                                                                                                                                                                                                                                                                     |
| Objective of the Test                     | Verify that the products endpoint is publicly accessible and returns a valid JSON array of products.                                                                                                                                                                                                 |
| Tool / Framework Used                     | Python 3.13, pytest, requests                                                                                                                                                                                                                                                                        |
| Steps / Procedure                         | <ol style="list-style-type: none"><li>1. Send a GET request to /products.php with no authentication header.</li><li>2. Assert that the HTTP response status is 200.</li><li>3. Assert that the response is a JSON list.</li></ol>                                                                    |
| Test Data / Input                         | No input — unauthenticated GET request                                                                                                                                                                                                                                                               |
| Expected Result                           | HTTP 200 with a JSON array containing product objects.                                                                                                                                                                                                                                               |
| Actual Result                             | HTTP 200 returned. Response confirmed as a list. 43 product(s) retrieved successfully.                                                                                                                                                                                                               |
| Test Status                               | [ PASSED ]                                                                                                                                                                                                                                                                                           |
| Screenshot / Evidence                     | <pre>test_products.py::test_get_products_returns_list   Sending GET /products.php with no authentication (public endpoint)...   Response status : 200   Checking: status is 200...   Checking: response is a list (not a dict or error)...   [PASS] GET products returned 43 product(s) PASSED</pre> |

| Test Case #02 — Unauthorized Product Creation |                                                                                                                                               |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Functionality Tested                          | Products POST — Authentication Enforcement (Unauthorized Access)                                                                              |
| Objective of the Test                         | Verify that the server correctly rejects any product creation request that does not include a valid authorization header, returning HTTP 401. |
| Tool / Framework Used                         | Python 3.13, pytest, requests                                                                                                                 |

|                              |                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Steps / Procedure</b>     | <ol style="list-style-type: none"> <li>1. Send a POST request to /products.php with a sample product payload but with no authorization header.</li> <li>2. Assert that the HTTP response status is 401.</li> </ol>                                                                                                                                                |
| <b>Test Data / Input</b>     | { "name": "Unauthorized Coffee", "category": "Coffee", "price": 80 } — no auth token                                                                                                                                                                                                                                                                              |
| <b>Expected Result</b>       | HTTP 401 Unauthorized — server refuses the request.                                                                                                                                                                                                                                                                                                               |
| <b>Actual Result</b>         | HTTP 401 returned. Server correctly rejected the unauthenticated product creation attempt.                                                                                                                                                                                                                                                                        |
| <b>Test Status</b>           | <b>[ PASSED ]</b>                                                                                                                                                                                                                                                                                                                                                 |
| <b>Screenshot / Evidence</b> |  <pre> test_products.py::test_post_product_requires_auth   Sending POST /products.php with NO Authorization header...   Response status : 401   Checking: server returns 401 (Unauthorized)...   [PASS] Unauthenticated product creation correctly rejected (401) PASSED </pre> |

| Test Case #03 — Create Valid Product |                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functionality Tested</b>          | Products POST — Successful Product Creation (Authenticated)                                                                                                                                                                                                                                                           |
| <b>Objective of the Test</b>         | Verify that an admin can successfully create a new product and that the server returns HTTP 201 along with the generated product ID.                                                                                                                                                                                  |
| <b>Tool / Framework Used</b>         | Python 3.13, pytest, requests                                                                                                                                                                                                                                                                                         |
| <b>Steps / Procedure</b>             | <ol style="list-style-type: none"> <li>1. Send a POST request to /products.php with the admin token and all required fields (name, category, price, sizes, temps, status).</li> <li>2. Assert that the HTTP response status is 200 or 201.</li> <li>3. Assert that the response body contains an id field.</li> </ol> |
| <b>Test Data / Input</b>             | { "name": "AutoTest Brew", "category": "Coffee", "price": 95, "sizes": ["Small", "Medium", "Large"], "temps": ["Hot"], "status": "available" } — with admin auth token                                                                                                                                                |
| <b>Expected Result</b>               | HTTP 201 with a response containing a generated product id.                                                                                                                                                                                                                                                           |
| <b>Actual Result</b>                 | HTTP 201 returned. Product created successfully with ID: PE7DBBFB0.                                                                                                                                                                                                                                                   |
| <b>Test Status</b>                   | <b>[ PASSED ]</b>                                                                                                                                                                                                                                                                                                     |

|                       |                                                                                                                                                                                                                                                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screenshot / Evidence | <pre> test_products.py::test_post_product_valid Sending POST /products.php with admin token and all valid fields... Product ID used : PE7DBBF0 Response status : 201 Response body : {'id': 'PE7DBBF0'} Checking: status is 200 or 201 and response contains 'id'... [PASS] Product created successfully: PE7DBBF0 PASSED </pre> |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### IV. Reflection & Findings

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Issues I Personally Encountered   | The API enforces a rate limit on admin login attempts (5 per 5 minutes). During early test runs, repeatedly running the script exhausted the rate limit quickly, causing subsequent tests to fail with HTTP 429 (Too Many Requests). I resolved this by coordinating with the group to add a localhost IP exemption in <i>rate_limit.php</i> , which is standard practice for local test environments.                            |
| Warnings / Errors I Detected      | I encountered a <i>PytestReturnNotNoneWarning</i> after noticing that some of my earlier test functions were returning unnecessary return statements. Since pytest expects test functions to return None, I removed those return statements to get rid of the warning.                                                                                                                                                            |
| Bugs I Discovered                 | No critical bugs were found in the Products API. All CRUD operations and authorization checks worked as expected during testing. However, after discussing with the group, we found that the <i>issueAdminToken()</i> function only supports one active token per staff member. This means that if an admin logs in on a second device, their existing session gets invalidated, which could lead to usability issues for admins. |
| What I Did About the Issues       | For the rate limit issue, I coordinated with the group lead to add the localhost exemption. For the <i>PytestReturnNotNoneWarning</i> , I removed all return statements from my test functions. I also used the session-scoped <i>admin_token</i> fixture from <i>conftest.py</i> to ensure only one login call is made for the entire test session, which avoids consuming rate-limit slots on each individual test.             |
| What I Learned from This Activity | I learned that automated API testing takes more planning than just sending requests over and over—you need to consider security measures like rate limiting and work within the server's limits. I also realized how useful shared fixtures in <i>conftest.py</i> can be. By centralizing the admin login into a session-scoped fixture, I was able to reduce                                                                     |

|  |                                                                                                                                                                                                                   |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | repetitive code and make the test suite faster and more reliable. More importantly, building these tests gave me a much deeper understanding of the Products API in a way that manual testing never really could. |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|